

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 1)

MAHALINGAM Nithees (exercise 2)

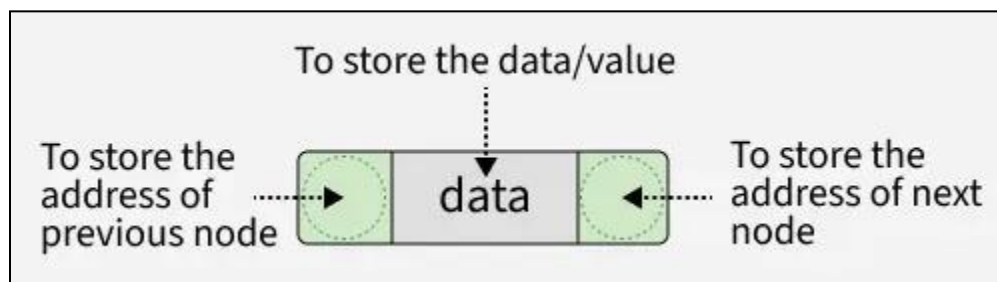
SARAVANAN Arun Prasath (exercise 3)

Github Link:

<https://github.com/anshuman-krishna/advanced-algorithms-and-programming>

Exercise 1: YADAV Anshuman Krishna

Content Feed Navigation with Doubly Linked List



1. StoryNode

```
STRUCTURE StoryNode
story_id : String
user_id : String
content_preview : String
timestamp : Integer
views : Integer
next : StoryNode
prev : StoryNode
END STRUCTURE
```

2. DoublyLinkedList

```
STRUCTURE DoublyLinkedList
head : StoryNode
tail : StoryNode
current : StoryNode
size : Integer
END STRUCTURE
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

3. Core

A. add_story

```
PROCEDURE add_story(list, node)
```

```
node.next <- NULL
```

```
node.views <- 0
```

```
IF
```

```
list.head = NULL THEN
```

```
node.prev <- NULL
```

```
list.head <- node
```

```
list.tail <- node
```

```
list.current <- node
```

```
ELSE
```

```
node.prev <- list.tail
```

```
list.tail.next <- node
```

```
list.tail <- node
```

```
END IF
```

```
list.size <- list.size + 1
```

```
END PROCEDURE
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

B. remove_story

```
PROCEDURE remove_story(list, story_id)
```

```
temp <- list.head
```

```
WHILE temp != NULL DO
```

```
IF temp.story_id = story_id THEN
```

```
IF temp.prev != NULL THEN
```

```
temp.prev.next <- temp.next
```

```
ELSE
```

```
list.head <- temp.next
```

```
END IF
```

```
IF temp.next != NULL THEN
```

```
temp.next.prev <- temp.prev
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
ELSE
list.tail <- temp.prev
END IF

IF list.current = temp THEN
list.current <- temp.next
END IF

list.size <- list.size - 1
Break
END IF
temp <- temp.next
END WHILE
END PROCEDURE
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

C. move_forward

```
FUNCTION move_forward(list) -> String
IF list.current != NULL AND list.current.next != NULL THEN
list.current <- list.current.next
RETURN list.current.content_preview
END IF
RETURN "End of feed"
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

D. move_backward

```
FUNCTION move_backward(list) -> String
IF list.current != NULL AND list.current.prev != NULL THEN
list.current <- list.current.prev
RETURN list.current.content_preview
END IF
RETURN "Beginning of feed"
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

E. **jump_to(story_id):**

```
PROCEDURE jump_to(list, story_id)
temp <- list.head
WHILE temp != NULL DO
IF temp.story_id = story_id THEN
list.current <- temp
Break
END IF
temp <- temp.next
END WHILE
END PROCEDURE
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

F. **insert_after(current_id, new_story)**

```
PROCEDURE insert_after(list, current_id, new_story)
temp <- list.head
WHILE temp != NULL DO
IF temp.story_id = current_id THEN
new_story.next <- temp.next
new_story.prev <- temp

IF temp.next != NULL THEN
temp.next.prev <- new_story
ELSE
list.tail <- new_story
END IF

temp.next <- new_story
list.size <- list.size + 1
Break
END IF
temp <- temp.next
END WHILE
END PROCEDURE
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

G. display_around_current(k)

...

to be continued in T24_LAB3_BFM

4. Engagement Tracking:

A. track_view(current)

```
PROCEDURE track_view(list)
IF list.current != NULL THEN
list.current.views <- list.current.views + 1
END IF
END PROCEDURE
```

B. most_viewed

Complexity Analysis Questions:

1. Compare time complexity of forward/backward navigation in singly vs doubly linked list?

Answer:

2. What is the time complexity of removing a node given only a pointer to that node?

Answer:

3. How much extra memory per node does a doubly linked list require compared to singly linked?

Answer:

4. In what scenarios would the bidirectional navigation justify the memory overhead?

Answer:

Exercise 2: MAHALINGAM Nithees

Activity Feed Processing with Stacks and Queues

Part A : Recent Activity Stack (LIFO)

Structure:

Structure Node

 activity

 next

End Structure

Structure ActivityStack

 top

 size

End Structure

Initialize Stack:

Function createStack()

 stack.top <- NULL

 stack.size <- 0

 return stack

End

Push:

Function push(stack, activity)

 newNode <- new Node

 newNode.activity <- activity

 newNode.next <- stack.top

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
stack.top <- newNode
stack.size <- stack.size + 1
End
```

Pop:

```
Function pop(stack)
  If stack.top = NULL
    return "Stack Empty"
  EndIf

  temp <- stack.top
  stack.top <- stack.top.next
  stack.size <- stack.size - 1

  return temp.activity
End
```

Peek:

```
Function peek(stack)
  If stack.top = NULL
    return "Stack Empty"
  EndIf

  return stack.top.activity
End
```

Is_empty:

```
Function is_empty(stack)
  If stack.top = NULL
    return TRUE
  Else
    return FALSE
  End
End
```

Size:

```
Function size(stack)
  return stack.size
End
```

Display_recent:

```
Function display_recent(stack)
  current <- stack.top
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
While current != NULL
  print current.activity
  current <- current.next
End
```

Undo operation:

```
Function undo(stack, undoStack)
  activity <- pop(stack)

  If activity != "Stack Empty"
    push(undoStack, activity)
  EndIf
End
```

Redo Operation:

```
Function redo(stack, undoStack)
  activity <- pop(undoStack)

  If activity != "Stack Empty"
    push(stack, activity)
  EndIf
End
```

Part B : Notification Queue (FIFO)

Structure:

```
Structure QNode
  notification
  next
End Structure
```

```
Structure NotificationQueue
  front
  rear
  size
End Structure
```

Enqueue:

```
Function enqueue(queue, notification)

  newNode <- new QNode
```


II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
newNode.notification <- notification
newNode.next <- NULL
```

```
If queue.rear = NULL
    queue.front <- newNode
    queue.rear <- newNode
Else
    queue.rear.next <- newNode
    queue.rear <- newNode
EndIf
```

```
queue.size <- queue.size + 1
End
```

Dequeue:

Function dequeue(queue)

```
If queue.front = NULL
    return "Queue Empty"
EndIf
```

```
temp <- queue.front
queue.front <- queue.front.next
```

```
If queue.front = NULL
    queue.rear <- NULL
EndIf
```

```
queue.size <- queue.size - 1
return temp.notification
End
```

Front:

Function front(queue)

```
If queue.front = NULL
    return "Queue Empty"
EndIf
```

```
return queue.front.notification
End
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Is_empty:

Function is_empty(queue)

```
If queue.front = NULL
    return TRUE
Else
    return FALSE
```

End

Size:

Function size(queue)

```
    return queue.size
```

End

Display_pending:

Function display_pending(queue)

```
    current <- queue.front
```

```
    While current != NULL
        print current.notification
        current <- current.next
```

End

Priority Enqueue:

Function priority_enqueue(queue, notification)

```
    newNode <- new QNode
    newNode.notification <- notification
```

```
    newNode.next <- queue.front
    queue.front <- newNode
```

```
    If queue.rear = NULL
        queue.rear <- newNode
    EndIf
```

```
    queue.size <- queue.size + 1
```

End

Part C : Integrated Feed Processing System

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Structure:

```
Structure FeedProcessor
  recent_stack
  notification_queue
  processed_log
End Structure
```

Process_notifications:

```
Function process_notifications(system)
  notification <- dequeue(system.notification_queue)
  If notification != "Queue Empty"
    push(system.recent_stack, notification)
  EndIf
End
```

Batch_process:

```
Function batch_process(system, k)
  For i <- 1 to k
    notification <- dequeue(system.notification_queue)
    If notification = "Queue Empty"
      break
    EndIf

    push(system.recent_stack, notification)
  EndFor
End
```

Clear_history:

```
Function clear_history(system)
  While NOT is_empty(system.recent_stack)
    activity <- pop(system.recent_stack)
    enqueue(system.processed_log, activity)
  EndWhile
End
```

Get_stats:

```
Function get_stats(system)
  print "Recent Activities:", size(system.recent_stack)
  print "Pending Notifications:", size(system.notification_queue)
  print "Processed Items:", size(system.processed_log)
End
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Complexity Analysis Questions:

1. Compare time complexity of all operations.

Answer:

Operation	Complexity
push()	$O(1)$
pop()	$O(1)$
peek()	$O(1)$
enqueue()	$O(1)$
dequeue()	$O(1)$
priority_enqueue()	$O(1)$
display_recent	$O(n)$
display_pending()	$O(n)$
batch_process()	$O(k)$

2. For a queue, which implementation (array circular buffer vs. linked list) is more memory efficient for unknown size?

Answer:

3. What is the amortized time complexity of a sequence of n enqueue/dequeue operations?

Answer:

4. How would you implement a deque (double-ended queue) to support both stack and queue operations?

Answer:

Exercise 3: SARAVANAN Arun Prasath

Engagement-Based Priority Queue

1. Post Structure

```
STRUCT Post
  post_id
  user_id
  content
  timestamp
  likes
  comments
  shares
  engagement_score
  next
END STRUCT
```

2. Calculate Engagement Score

```
FUNCTION calculate_score(post)
  score <- (post.likes * 1) + (post.comments * 2) + (post.shares * 3)
  RETURN score
END FUNCTION
```

3. Initialize Priority Queue

```
STRUCT PriorityQueue
  head <- NULL
  count <- 0
END STRUCT
```

4. Enqueue (Insert Post in Sorted Order)

```
FUNCTION enqueue(queue, new_post)
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
new_post.engagement_score <- calculate_score(new_post)
IF queue.head = NULL OR
  new_post.engagement_score > queue.head.engagement_score THEN
  new_post.next <- queue.head
  queue.head <- new_post
ELSE

  current <- queue.head
  WHILE current.next != NULL AND
    current.next.engagement_score ≥ new_post.engagement_score DO
    current <- current.next
  END WHILE

  new_post.next <- current.next
  current.next <- new_post

END IF
queue.count <- queue.count + 1
END FUNCTION
```

5. Dequeue Max (Remove Highest Priority)

```
FUNCTION dequeue_max(queue)

  IF queue.head = NULL THEN
    RETURN NULL
  END IF

  temp <- queue.head
  queue.head <- queue.head.next

  queue.count <- queue.count - 1

  RETURN temp

END FUNCTION
```

6. Peek Max (View Highest Priority)

```
FUNCTION peek_max(queue)

  IF queue.head = NULL THEN
    RETURN NULL
```

```
END IF
```

```
RETURN queue.head
```

```
END FUNCTION
```

7. Check if Queue is Empty

```
FUNCTION is_empty(queue)
```

```
    IF queue.head = NULL THEN
```

```
        RETURN TRUE
```

```
    ELSE
```

```
        RETURN FALSE
```

```
    END IF
```

```
END FUNCTION
```

8. Get Queue Size

```
FUNCTION size(queue)
```

```
    RETURN queue.count
```

```
END FUNCTION
```

9. Update Post Score

```
FUNCTION update_score(queue, post_id, new_likes, new_comments, new_shares)
```

```
    current <- queue.head
```

```
    previous <- NULL
```

```
    WHILE current != NULL DO
```

```
        IF current.post_id = post_id THEN
```

```
            IF previous = NULL THEN
```

```
                queue.head <- current.next
```

```
            ELSE
```

```
                previous.next <- current.next
```

```
            END IF
```

```
            queue.count <- queue.count - 1
```

```
            current.likes <- new_likes
```

```
            current.comments <- new_comments
```

```
current.shares <- new_shares
```

```
enqueue(queue, current)
```

```
RETURN
```

```
END IF
```

```
previous <- current
```

```
current <- current.next
```

```
END WHILE
```

```
END FUNCTION
```

10. Get Top K Posts

```
FUNCTION get_top_k(queue, k)
```

```
current <- queue.head
```

```
i <- 0
```

```
WHILE current != NULL AND i < k DO
```

```
    PRINT current.post_id, current.engagement_score
```

```
    current <- current.next
```

```
    i <- i + 1
```

```
END WHILE
```

```
END FUNCTION
```

11. Decay Older Posts

```
FUNCTION decay_older_than(queue, time)
```

```
current <- queue.head
```

```
WHILE current != NULL DO
```

```
    IF current.timestamp < time THEN
```

```
        current.engagement_score <- current.engagement_score * 0.8
```

```
    END IF
```

```
    current <- current.next
```


II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

END WHILE

END FUNCTION

Complexity Analysis Questions:

1. Compare time complexity of enqueue for:

Answer:

- **Sorted linked list implementation:** $O(n)$. To keep the list sorted, you must traverse it from the head to find the correct insertion point. In the worst case (inserting a very low score), you visit every node.
- **Unsorted linked list with find-max each time:** $O(1)$. If the list is unsorted, you can simply "push" the new node to the front of the list. (Note: While enqueue is fast, the corresponding dequeue_max would become $O(n)$ because you'd have to search the whole list for the highest score).

2. Worst-case time complexity of update_score? How could you optimize it?

Answer:

3. For a social network with millions of posts, would a sorted linked list be practical?

Answer:

4. Space complexity: How much extra memory per node?

Answer:

5. If engagement scores update frequently, what data structure would be more efficient?

Answer:

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

...

to be continued in T24_LAB3_BFM